

Editor setup and requirements

What we need in place to build and run the data editor: how analysts will sign in, why we need a paid GitHub plan, and the order we should set things up in.

Repo: Keith-paradox/launch-development (private) Audience: internal
Status: current as of 16 Sep 2026

1. What we are building

A private web page where an analyst updates the dashboard's information using forms instead of editing a data file by hand. It lives in the same repository as the dashboard itself, so both hand over to RBM as a single item later.

The editor does not publish anything directly. It saves a draft, the draft is checked automatically, someone looks at a preview, and only then is it published. GitHub provides the storage, the history, the identity checks and the approval step – most of which this repository already uses today.

2. Where we are today

ITEM	CURRENT STATE
Repository	Private, owned by the Keith-paradox personal account
GitHub plan	Free
Automatic data checks	Working – validate.yml runs on every push and pull request
History, RSS feed, data exports	Working – publish.yml runs on merge to main
Rules protecting the main branch	Not available on the current plan

GitHub reports this directly when we query the repository:

Upgrade to GitHub Pro or make this repository public to enable this feature.

Note also that the KylerXiv account has push access but not admin. Plan changes, branch rules and the eventual transfer all have to be done from Keith-paradox.

3. How analysts will sign in

Using a GitHub token. Each person creates their own – we do not build a login system, store passwords, or handle password resets. GitHub does all of that.

How it works

- The analyst creates a **fine-grained token** in their GitHub account settings
- They paste it into the editor once
- The editor uses it to read the data, save drafts and open them for approval

How the token is limited

SETTING	VALUE	REASON
Repository access	This repository only	A leak cannot reach anything else
Permissions	Contents, and pull requests	No admin, no settings, no workflows
Expiry	90 days	Tokens that live forever get forgotten
Where it is stored	<code>sessionStorage</code>	Gone when the tab closes; never written to disk

What the token is not

- It is **not** built into the editor page, and not in the published files
- It is **not** shared between people – each analyst has their own
- The editor page holds **no secret of its own**, so there is nothing in it worth stealing

Every change therefore carries a real person's name, and access can be revoked for one person without affecting anyone else. GitHub logs every use of every token.

Later: one-click sign in

We can replace the paste step with a **Sign in with GitHub** button by registering a GitHub App. It issues short-lived tokens that expire after an hour, scoped to this repository. It needs one small server-side function, so it depends on the hosting decision, which is RBM's to make.

Starting with pasted tokens costs us nothing later: only the part that *obtains* the token changes. Everything downstream is untouched.

4. Why we need GitHub Pro

Token permissions cannot be limited to a branch

A token that can write files can write to `main`. There is no setting for "can save drafts but cannot publish" – permissions apply to the whole repository.

Branch protection is the only thing that enforces the difference. Without it, "the editor can only propose changes" is a convention we are following in our own code, not something GitHub prevents.

With Pro we can set two rules on `main`:

- **No direct pushes.** Changes must arrive as a draft.
- **The data check must pass before merging.** `validate.yml` already runs; this makes passing it mandatory rather than advisory.

Together these make the security model real: a stolen or misused token can open a draft that nobody approves, and nothing more.

Pro also allows publishing a public page from a private repository, which is useful but not the reason to upgrade.

Cost

WHEN	PLAN	COST
Now – while we build	GitHub Pro on <code>Keith-paradox</code>	\$4 / month total
After handover	GitHub Team on RBM's organisation	\$4 / person / month
Expected at handover	3 editors	about \$12 / month

5. Setting it up, in order

PHASE 1 Build — no payment needed

Everything in the editor can be built and tested on the Free plan. Only enforcement is missing, and that does not block development.

- Extract the data rules into a shared module so the editor and the command-line checker use the same ones. This blocks everything else – start here.
- Port the file-saving format to JavaScript so saved files match the repo's existing style and produce clean diffs.
- Build the editor page: forms, live checking, the changelog requirement.
- Build the preview – the real dashboard page in a frame, fed the draft data. Instant, and needs no hosting.
- Connect to GitHub: read the data, create a branch, save the file, open it for approval.

While building: point the editor at a scratch branch and do not wire the publish button. Without branch protection, a write token can reach `main`, so we keep it out of reach by design until Phase 2.

Running it locally needs no hosting at all:

```
python3 -m http.server 8001 --bind 127.0.0.1
# open http://localhost:8001/editor.html
```

Use a local server rather than opening the file directly – GitHub's API rejects requests from a `file://` page. **--bind 127.0.0.1 is not optional:** without it `http.server` listens on every network interface and serves the whole private repo to anyone who can reach the machine.

PHASE 2 Turn on the protections — \$4

- Upgrade `Keith-paradox` to GitHub Pro.
- Add a rule on `main`: no direct pushes, and the data check must pass before merging.
- Switch the editor from the scratch branch to real draft branches.
- Wire the publish button. It is only safe once the rules above exist.
- Confirm the whole loop end to end – including that the history snapshot still runs when a change arrives by merge rather than a direct push.

PHASE 3 Handover

- Create an RBM organisation and transfer the repository into it. History, people and automation move with it.
- RBM subscribes to GitHub Team so the same branch rules apply.
- Add RBM's editors; set up two groups – one that can save drafts, one that can also publish.
- Add the dashboard to the RBM website.
- Hand over the analyst guide.

6. Still to confirm with RBM

None of these block Phases 1 or 2.

QUESTION	WHAT IT AFFECTS
Can their staff have GitHub accounts?	If not, we need a work-email sign-in with a shared machine account – weaker attribution and more to build
Will their site display our page, or copy the files?	Whether publishing stays automatic after handover
Who owns the accounts and hosting?	Where we create the GitHub App and hosting – cheaper to get right first time
Is endmalaria.org run in-house or by a contractor?	Who we are actually handing over to

7. Summary

REQUIREMENT	COST	WHEN
A GitHub account per editor	Free	Before they start editing
A fine-grained token per editor	Free	Before they start editing
GitHub Pro, for the branch rules	\$4 / month	Before the publish button is wired
An RBM organisation	Free	At handover
GitHub Team for RBM	\$4 / person / month	At handover
Somewhere to serve the pages	Free to ~\$20 / month	RBM's decision